FIELD GUIDE · AGENT OBSERVABILITY

The 12 ways AI agents fail that your logs will never show you.

Your dashboard is green. Status codes are 200. And something just went very wrong.

- API 200 OK
- latency p99 ok
- error rate 0%
- trajectory: ???

01

MCP tool-description rug-pulls

A tool can rewrite its own description **between one call and the next**. The agent reasons over “returns the weather” — and runs something else.

REQUEST LOG

LOGGED	✓	tool_call → 200 OK
REALITY	✗	the **contract the agent trusted** was swapped mid-flight

BLIND SPOT Logs capture the call, not the definition the agent reasoned over.

02

Lethal-trifecta exfiltration

Private data + untrusted content + a way to send messages = **an exfiltration machine**. Every single step is authorized.

REQUEST LOG

LOGGED	✓ read inbox ✓ fetch url ✓ send msg ✓
REALITY	✗ data **walked out across three legal actions**

BLIND SPOT A per-request log has no concept of a path.

03

Browser-agent stuck loops

The page didn’t transition the way the model expected. So it clicks again. And again. **Eleven times.** It all reads as activity.

REQUEST LOG

LOGGED	✓	requests flowing · latency normal
REALITY	✗	**same action, same result, 11×**

BLIND SPOT No single request is anomalous. The pattern is the anomaly.

04

Agent-to-agent handoff breaks

A field gets renamed upstream. The receiving agent doesn’t error — it **hallucinates a plausible value** and proceeds confidently.

REQUEST LOG

LOGGED	✓	handoff complete · no exception thrown
REALITY	✗	**corruption entered at the seam**

BLIND SPOT Request logs don’t instrument the seams between agents.

05

Prompt-injection cascades

One poisoned input **reshapes every step after it.** The instruction rides along in context, steering tool choices many calls later.

REQUEST LOG

LOGGED	✓	each later call → healthy, independent
REALITY	✗	a **contaminated chain of decisions**

BLIND SPOT The causal chain you need for an incident review isn’t there.

06

Hallucinated function calls

The model calls a tool that doesn’t exist — or invents arguments that were never valid — and **the framework runs it anyway**.



BLIND SPOT Intent is exactly the thing logs don’t store.

07

Cost drift on long-running agents

A retry loop. An ever-growing context. The same doc re-read every iteration. **Each call costs** a fraction of a cent.

REQUEST LOG

LOGGED	✓	per-call cost negligible
REALITY	✗	the count **grows quietly, all day**

BLIND SPOT You discover it from the provider invoice, not the dashboard.

08

Jailbreak attempts

Not one obvious message.
Assembled across turns,
buried mid-conversation,
smuggled inside tool output the
agent trusts.



BLIND SPOT The same phrase is benign in one position, an attack in another.

09

PII leakage into spans

You capture inputs and state as traces to debug — then ship them to a third-party vendor. **An** SSN goes with them.

REQUEST LOG

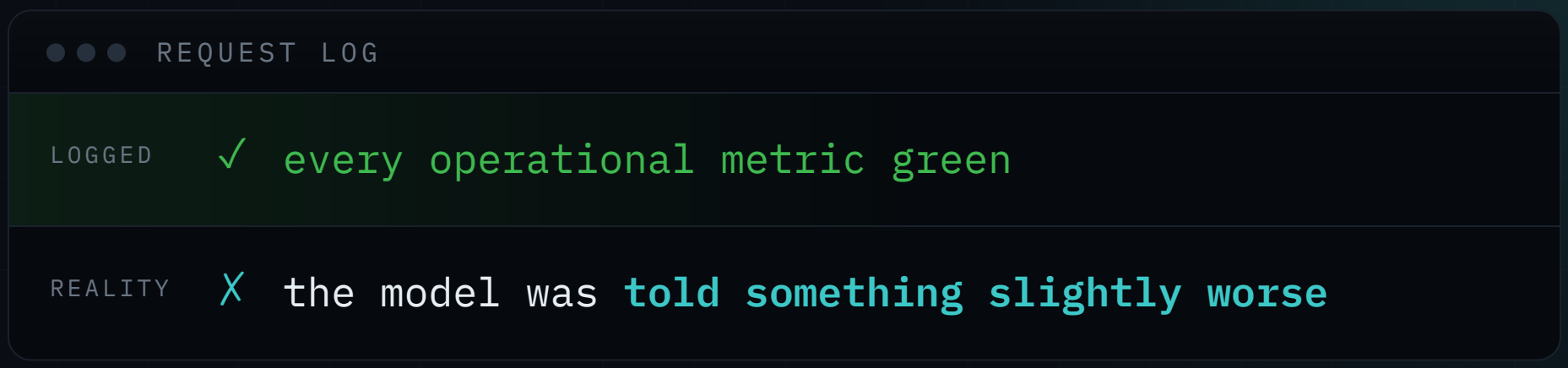
LOGGED	✓	agent worked perfectly
REALITY	✗	the **leak is in your telemetry**

BLIND SPOT Redaction has to happen before the span leaves your boundary.

10

Eval regression on prompt updates

Someone tightens a prompt and deploys. Latency flat. Errors flat. **Output quality** silently degrades.



BLIND SPOT No infrastructure metric measures “worse.”

11

Model-version drift

The provider improves the model behind the same API name. Your code didn’t change. Your prompt didn’t change. **The** behavior did.

REQUEST LOG

LOGGED	✓	identical requests · 200s · normal latency
REALITY	✗	**behavior drifted out from under you**

BLIND SPOT Logs have no memory of yesterday’s behavior to compare against.

12

Region-pinned compliance violations

EU data must stay in EU regions. An outage triggers failover. The request is served from somewhere it was never allowed.

REQUEST LOG

LOGGED ✓ failover worked · uptime preserved

REALITY ✗ a **legal boundary was crossed**

BLIND SPOT Uptime monitoring celebrates the exact event that caused it.

The failure lives between steps, across steps, or in the meaning of a step — never in the step itself.

Conventional logging instruments the step. It’s a **stopwatch measuring temperature** — not bad, just measuring something else.

```
→ Treat the trajectory as the unit of observability.
If your tooling sees fewer than 12, know which ones you’re
blind to.
```